

实验九实验报告

姓名： 林宏宇 学号： 23320093

1. 实验内容

5.1 实验总体框架及单周期处理器连线图

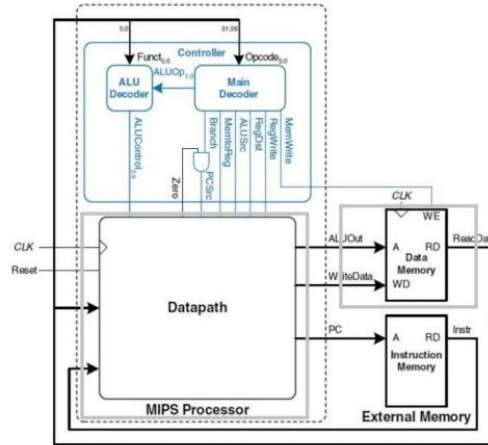


图 1: 单周期 CPU 框架图

如图 1，完整的单周期 CPU 实现框架图。图中有些部分我们之前已经在实验 5.6,7 中已经完成（参见 4.2 中的描述），继续完成 datapath 模块，即可将单周期 MIPS 软核完整实现。

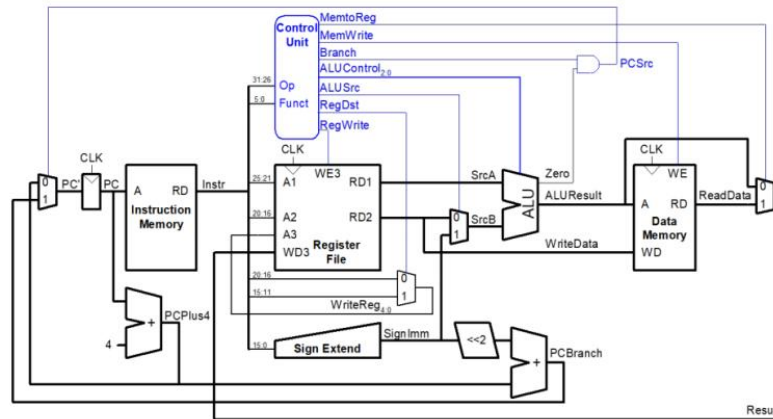


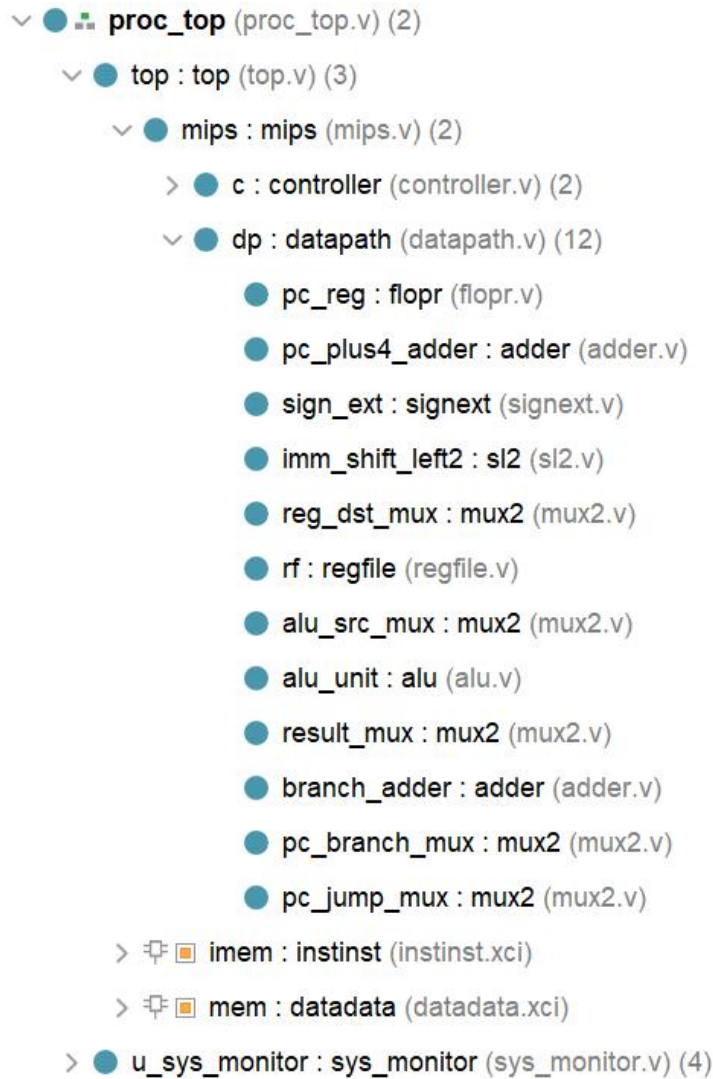
图 2: 单周期 CPU 系统连线图

图 2 为单周期的完整系统连线图，涵盖 *add*、*and*、*sub*、*or*、*slt*、*beq*、*j*、*lw*、*sw*、*addi* 等指令。本次实验仅实现上述几类指令，完整的 MIPS 指令集将与硬件综合设计中完善，此处以掌握数据通路分析为主要目的。

实现单周期流程，需要实现 coe 文件中指令的执行，并且能够通过串口进行监视

2. 实验过程

1. 添加代码模板、添加 ip 核；代码模块结构：



2. 实现 mux2 的代码, 根据单周期流程图 reg_dst、alu_src、result、pc_branch、pc_jump 都需要多路选择器进行选择。代码根据输入的 s 进行判断输出。

D:/Singal_cycle/MIPS_core/rtl/mux2.v

```

1 | timescale 1ns / 1ps
2 |
3 | module mux2 #(parameter WIDTH = 8) (
4 |     input wire[WIDTH-1:0] d0, d1,
5 |     input wire s,
6 |     output wire[WIDTH-1:0] y
7 | );
8 |
9 | //add your code here
0 | ○ assign y = s ? d1 : d0; // 如果s为1, 输出d1, 否则输出d0
1 | //code end
2 | endmodule

```

3. 实现 datapath，按照单周期的流程图进行例化和接信号。代码例化根据各个模块的代码进行例化，同时添加各个多路选择器 Mux 的例化。同时写转移地址 pcjump 和写回值 writedata。

```
20 //信号声明
21 wire [31:0] pcnext, pcplus4, pcbranch, pcjump;
22 wire [31:0] rd1, rd2, signimm, sl2_out, alu_input_b, result;
23 wire [4:0] writereg;
24
25 //PC寄存器
26 flopr #(32) pc_reg (
27     .clk(clk),
28     .rst(rst),
29     .d(pcnext),
30     .q(pc)
31 );
32
33 //PC + 4
34 adder pc_plus4_adder (
35     .a(pc),
36     .b(32'd4),
37     .y(pcplus4)
38 );
39
40 //符号扩展
41 signext sign_ext (
42     .a(instr[15:0]),
43     .y(signimm)
44 );
```

```

46 //左移2位
47 sl2 imm_shift_left2 (
48     .a(signimm),
49     .y(sl2_out)
50 );
51
52 //目标寄存器选择 (MUX)
53 mux2 #(5) reg_dst_mux (
54     .d0(instr[20:16]), //rt
55     .d1(instr[15:11]), //rd
56     .s(regdst),
57     .y(writereg)
58 );
59
60 //寄存器文件
61 regfile rf (
62     .clk(clk),
63     .we3(regwrite),
64     .ra1(instr[25:21]),
65     .ra2(instr[20:16]),
66     .wa3(writereg),
67     .wd3(result),
68     .rd1(rd1),
69     .rd2(rd2),
70     .ra_debug(ra_debug),
71     .ra_debug_data(ra_debug_data)
72 );
73
74 //ALU 输入选择 (MUX)
75 mux2 #(32) alu_src_mux (
76     .d0(rd2),
77     .d1(signimm),
78     .s(alusrc),
79     .y(alu_input_b)
80 );
81
82 //ALU
83 alu alu_unit (
84     .a(rd1),
85     .b(alu_input_b),
86     .op(alucontrol),
87     .y(aluout),
88     .overflow(overflow),
89     .zero(zero)
90 );
91
92 //数据写回寄存器选择 (MUX)
93 mux2 #(32) result_mux (
94     .d0(aluout),
95     .d1(readdata),
96     .s(memtoreg),
97     .y(result)
98 );
99

```

```

100 //分支地址计算
101 adder branch_adder (
102     .a(pcplus4),
103     .b(s12_out),
104     .y(pcbranch)
105 );
106
107 //跳转地址计算
108 assign pcjump = {pcplus4[31:28], instr[25:0], 2'b00};
109
110 wire [31:0] pcbranch_or_plus4;
111
112 //分支地址选择逻辑
113 mux2 #(32) pc_branch_mux (
114     .d0(pcplus4),
115     .d1(pcbranch),
116     .s(pcsrc),
117     .y(pcbranch_or_plus4) //输出给中间信号
118 );
119
120 //跳转地址选择逻辑
121 mux2 #(32) pc_jump_mux (
122     .d0(pcbranch_or_plus4),
123     .d1(pcjump),
124     .s(jump),
125     .y(pcnext) //最终输出给 pcnext
126 );
127
128
129 //输出写数据
130 // SW指令判断信号
131 wire isSW;
132 assign isSW = (instr[31:26] == 6'b101011); // opcode 为 101011 时是 SW 指令
133 assign writedata = isSW ? rd2 : 32'b0; // 仅在 SW 指令时输出 rd2, 否则置 0
134
135
136 endmodule

```

4. 仿真过程中发现模板中的 rom 和 ram 有问题，重新修改

```

20 //inst_mem 实例化
21 instinst imem (
22     .clka(clk), //时钟信号
23     .addra(pc[7:2]), //地址输入 (截取 pc 的低位地址)
24     .douta(instr), //输出数据 (指令)
25     .ena(1'b1) //使能信号恒为高电平
26 );
27
28 //修改后的 data_mem 实例化, 增加 ena 和 wea 信号
29 datadata mem (
30     .clka(clk), //时钟信号 (反向时钟)
31     .wea(memwrite), //写使能信号 (确保 memwrite 是 1 位信号)
32     // .addra(dataadr[7:2]), //地址输入 (截取 dataadr 的低位地址)
33     .addra({3'b0, dataadr[7:2]}), //扩展高3位为0
34     .dina(writedata), //输入数据 (写入数据)
35     .douta(readdata), //输出数据 (读取数据)
36     .ena(1'b1) //使能信号恒为高电平
37 );

```

5. 上板时发现 fifo 的例化和 fido 的接口不匹配，对串口进行调整。

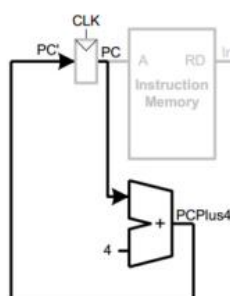
```
// clock domain convert
r2u_data_fifo u_run_2_uart_clock (
    .clk(clk_run_i), // input wire wr_clk
    .srst(~rst_n), // input wire rst
    // .rd_clk(clk_i), // input wire rd_clk
    .din(r2u_fifo_wdata), // input wire [63 : 0] din
    .wr_en(r2u_fifo_wren), // input wire wr_en
    .rd_en(r2u_fifo_rden), // input wire rd_en
    .dout(r2u_fifo_rdata), // output wire [63 : 0] dout
    .full(r2u_fifo_full), // output wire full
    .empty(r2u_fifo_empty) // output wire empty
);

u2r_cmd_fifo u_u2r_cmd_convert (
    .clk(clk_i), // input wire wr_clk
    .srst(~rst_n), // input wire rst
    .din(u2r_fifo_wdata), // input wire [7 : 0] din
    .wr_en(u2r_fifo_wren), // input wire wr_en
    .rd_en(u2r_fifo_rden), // input wire rd_en
    .dout(u2r_fifo_rdata), // output wire [7 : 0] dout
    .full(u2r_fifo_full), // output wire full
    .empty(u2r_fifo_empty) // output wire empty
);
// clock domain: clk_i = 50Mhz
```

3. 结果分析

1.数据通路分析，即 datapath 文件的实现原理。

①指令自加模块；使用单独的加法器来模拟 pc+4 自动更新



②lw 和 sw 指令：LW 指令的汇编格式为：LW rt, offset(base); 需要进行取指令、译码、alu 计算、lw 寄存器堆写回，如下图所示。

取值：

将 PC(即指令地址) 输出到 Instruction Memory(指令存储器) 的 address 端口；

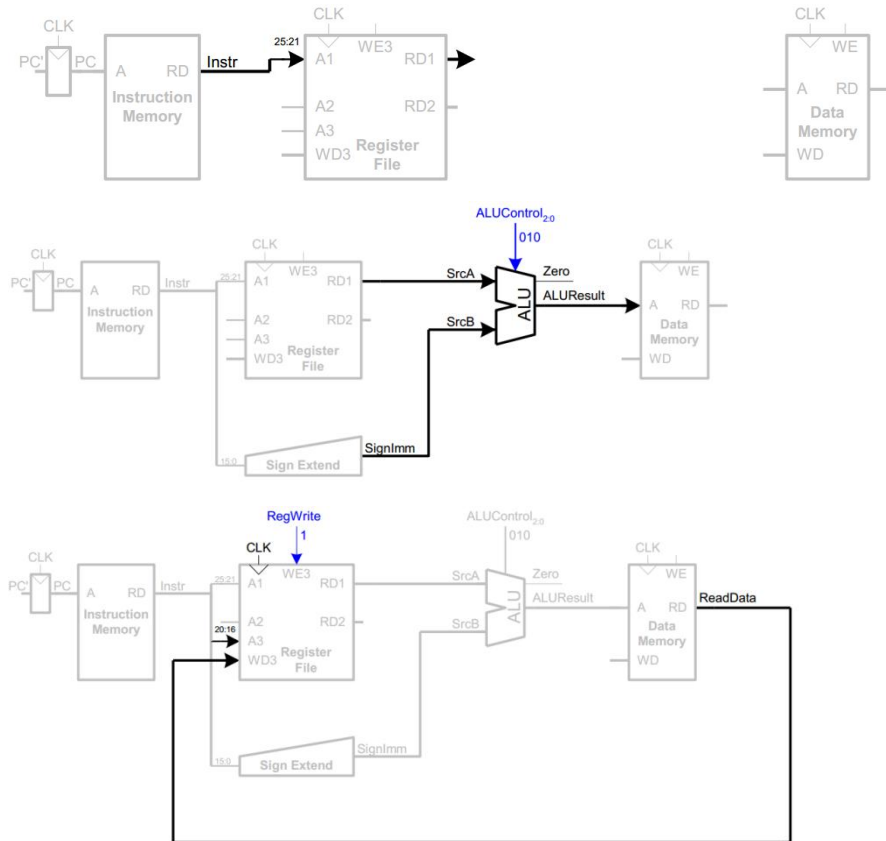
译码：

l, 指令 ([31:0]) 中 [25:21] 连接至 regfile 寄存器堆的第一个输入地址，即 Address1(A1)。

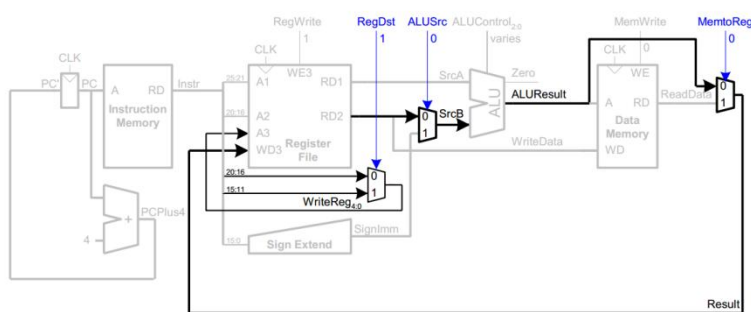
II, 需要将 offset 进行扩展, 因而将指令[15:0]传至有符号扩展模块, 输出 32 位的符号扩展信号 (SignImm)。

III, $base + sign_extend(offset)$, 其中加法计算使用 ALU。

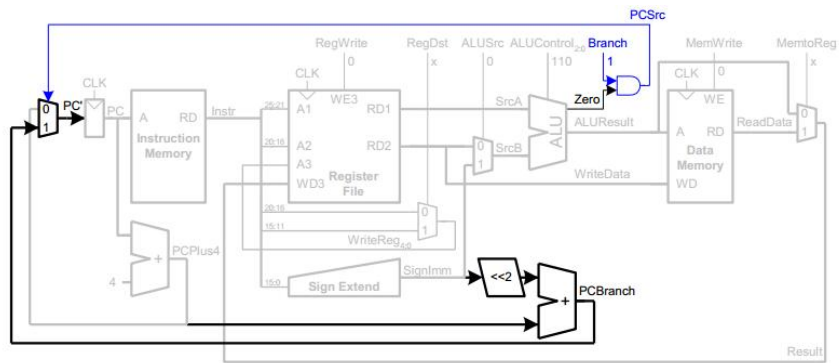
IV, 数据存储器读出的数据写回到 regfile 中, 其地址为 rt, 即指令的 [20:16]



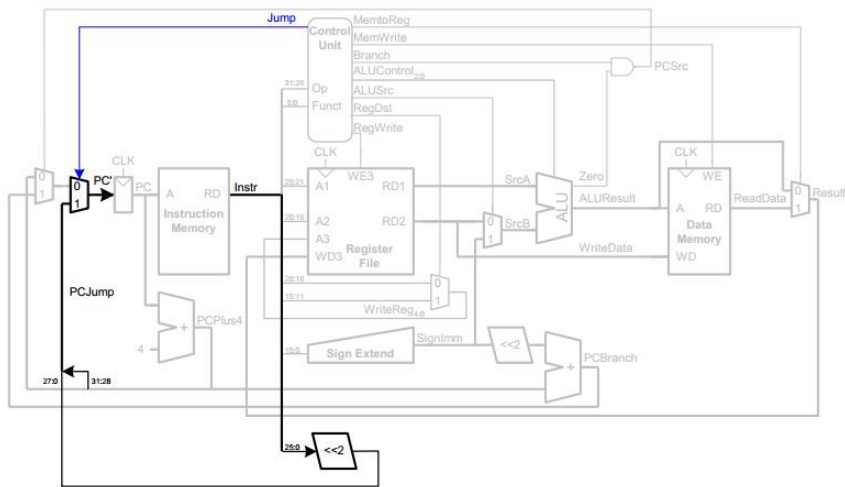
③R-type 指令: lw 指令写入 regfile 的地址为 rt, 而 R-Type 为 rd, 在此处加入一个多路选择器, 写回寄存器堆的数据是从 ALU 过来, 而不是 LW 操作中是从存储器过来, 因此要添加多路选择器。



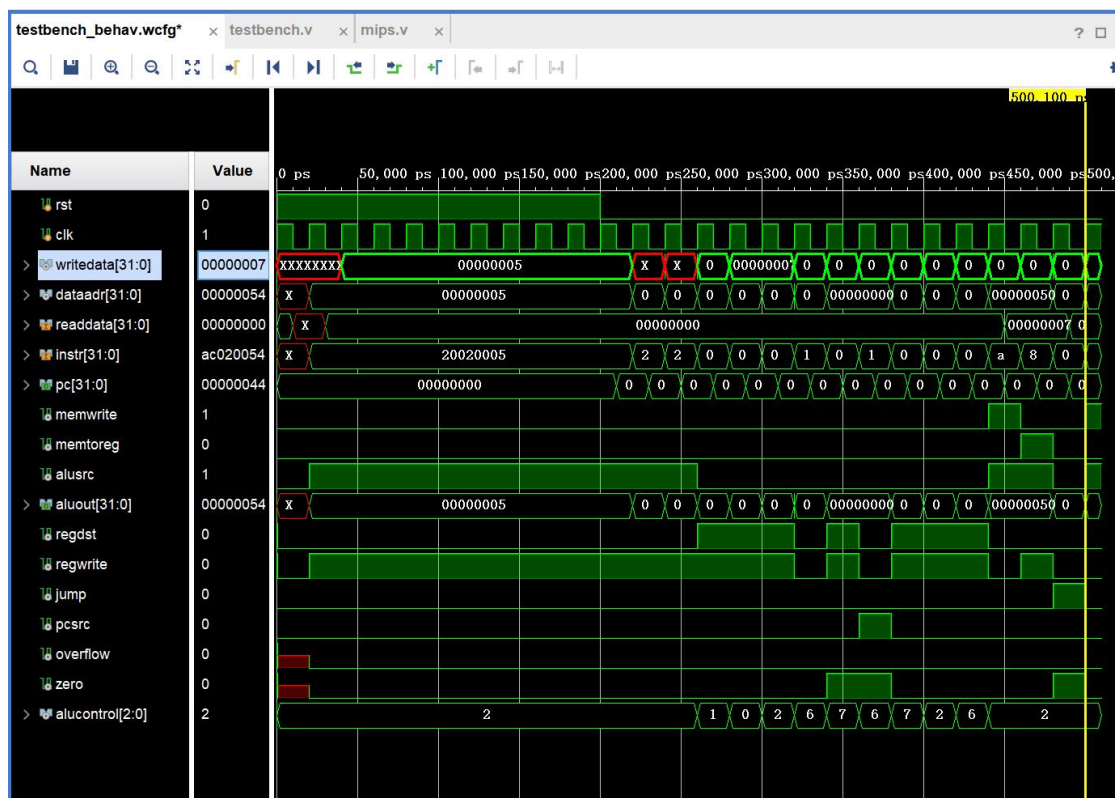
④Branch 指令: 条件判断使用 alu 进行算术计算, 偏移计算 $PC + 4 + sign_extend(offset) * 4$, PC 转移根据 PCSrc 信号进行控制, 满足条件时, PCSrc 为 1。



⑤J 指令：无条件跳转，不需要做任何判断，需要计算地址：PC(PC+4) 的最高 4 位与立即数 $instr_index(instr[25:0])$ 左移 2 位后的值拼接。



2.仿真结果



#	Assembly	Description	Address	Machine	
main:	addi \$2, \$0, 5	# initialize \$2 = 5	0	20020005	20020005
	addi \$3, \$0, 12	# initialize \$3 = 12	4	2003000c	2003000c
	addi \$7, \$3, -9	# initialize \$7 = 3	8	2067fff7	2067fff7
	or \$4, \$7, \$2	# \$4 <= 3 or 5 = 7	c	00e22025	00e22025
	and \$5, \$3, \$4	# \$5 <= 12 and 7 = 4	10	00642824	00642824
	add \$5, \$5, \$4	# \$5 = 4 + 7 = 11	14	00a42820	00a42820
	beq \$5, \$7, end	# shouldn't be taken	18	10a7000a	10a7000a
	slt \$4, \$3, \$4	# \$4 = 12 < 7 = 0	1c	0064202a	0064202a
	beq \$4, \$0, around	# should be taken	20	10800001	10800001
	addi \$5, \$0, 0	# shouldn't happen	24	20050000	20050000
around:	slt \$4, \$7, \$2	# \$4 = 3 < 5 = 1	28	00e2202a	00e2202a
	add \$7, \$4, \$5	# \$7 = 1 + 11 = 12	2c	00853820	00853820
	sub \$7, \$7, \$2	# \$7 = 12 - 5 = 7	30	00e23822	00e23822
	sw \$7, 68(\$3)	# [80] = 7	34	ac670044	ac670044
	lw \$2, 80(\$0)	# \$2 = [80] = 7	38	8c020050	8c020050
	j end	# should be taken	3c	08000011	08000011
	addi \$2, \$0, 1	# shouldn't happen	40	20020001	20020001
end:	sw \$2, 84(\$0)	# write adr 84 = 7	44	ac020054	ac020054

结合程序图和仿真波形图进行分析：

结果分析 end 处（即第 18 步）：当 address=44 时，执行到最后一步，此时 instr=ac020054、writedata=7、dataaddr=84（十进制）、memwrite=1；与程序最后一步的 write adr 84=7 的值相似，因此可以证明程序执行正确。

逐步分析信号过程：

根据 pc 与 instr 的变化进行信号分析

序号：指令

描述

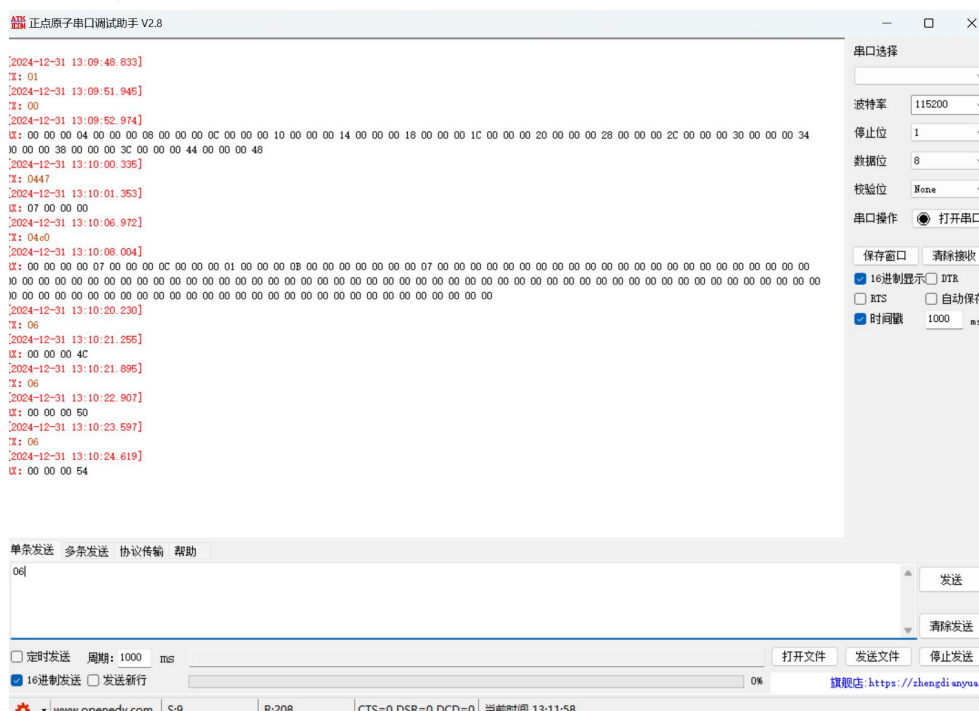
关键信号值检查

1: add \$2, \$0, 5

$\$2 = 5$
 检查 pc=0x00, instr=20020005, writedata=5, rd1=0, rd2=0。
 2: add \$3, \$0, 12
 $\$3 = 12$
 检查 pc=0x04, instr=2003000C, writedata=12, rd1=0, rd2=0。
 3: add \$7, \$3, -9
 $\$7 = 12 - 9 = 3$
 检查 pc=0x08, instr=2067FFF7, rd1=12, rd2=-9, aluout=3。
 4: or \$4, \$7, \$2
 $\$4 = \$7 \ \$2 = 3$
 5: and \$5, \$3, \$4
 $\$5 = \$3 \ \& \ \$4 = 12 \ \& \ 7 = 4$
 检查 pc=0x10, instr=00642824, rd1=12, rd2=7, aluout=4。
 6: add \$5, \$5, \$4
 $\$5 = \$5 + \$4 = 4 + 7 = 11$
 检查 pc=0x14, instr=00A42820, rd1=4, rd2=7, aluout=11。
 7: beq \$3, \$7, end
 条件不满足, 继续执行
 检查 pc=0x18, instr=10A7000A, rd1=12, rd2=3, pcbranch 未跳转。
 8: slt \$4, \$3, \$4
 $\$4 = (\$3 < \$4) ? 1 : 0 = 0$
 检查 pc=0x1C, instr=0064202A, rd1=12, rd2=7, aluout=0。
 9: beq \$4, \$0, around 条件满足, 跳转到 around 检查 pc=0x20, instr=10800001, rd1=0, rd2=0, pcbranch=around。
 10: add \$5, \$0, 0
 $\$5 = 0$
 检查 pc=0x24, instr=20050000, writedata=0, aluout=0。
 11: slt \$3, \$3, \$5
 $\$3 = (\$3 < \$5) ? 1 : 0 = 0$
 检查 pc=0x28, instr=0062202A, rd1=12, rd2=0, aluout=0。
 12: add \$7, \$3, \$5
 $\$7 = \$3 + \$5 = 0 + 0 = 0$
 检查 pc=0x2C, instr=00853820, rd1=0, rd2=0, aluout=0。
 13: sub \$7, \$7, \$5
 $\$7 = \$7 - \$5 = 0 - 0 = 0$
 检查 pc=0x30, instr=00E23822, rd1=0, rd2=0, aluout=0。
 14: sw \$7, 68(\$3)
 将 $\$7=0$ 写入地址 $\$3+68=80$
 检查 pc=0x34, instr=AC670044, dataadr=80, writedata=0, memwrite=1。
 15: lw \$2, 80(\$0)
 从地址 $\$0+80$ 读取值到 \$2
 检查 pc=0x38, instr=8C020050, dataadr=80, writedata=7, memwrite=0。
 16: j end
 跳转到 end

检查 pc=0x3C, instr=08000011, pcjump=end。
17: add \$2, \$0, 1
 $\$2 = \$0 + 1 = 1$
检查 pc=0x40, instr=20020001, writedata=1, aluout=1。
18: sw \$2, 84(\$0)
将 \$2=1 写入地址 \$0+84
检查 pc=0x44, instr=AC020054, dataadr=84, writedata=1, memwrite=1。

3.上板验证



1. 输入 01 复位
2. 输入 00 运行
3. 输入 0447, 即 00000100 01001011 (二进制) 可以查看一个寄存器的值
4. 输入 04c0, 可以查看全部寄存器的值
5. 输入三次 06, 可以一次得到当前 pc 的值, 且逐渐+4, 逻辑正确

通过串口测试, 可以证明实验正确无误。

4. 总结

1.掌握了单周期 CPU 的设计与实现方法。通过模块化设计, 将 ALU、寄存器堆、控制器、指令存储器和数据存储器等核心模块进行整合, 并通过顶层模块 proc_top 实现了完整的单周期 CPU 数据通路。实验过程帮助加深了对各模块功能和交互方式的理解。2.在仿真与上板测试过程中, 通过执行指令并监视寄存器值, 明确了指令解码、执行和结果存储的全过程。这一过程进一步巩固了对指

令执行流的理解。2.本次实验使用了 Xilinx Vivado 开发环境，从模块设计到 Block Memory 的初始化，再到顶层模块的连接与仿真、上板测试等环节，全面提升了 Vivado 工具的操作能力，为后续复杂设计奠定了基础。3.在仿真与实际测试中，通过分析波形图和寄存器值，对模块连接、数据路径和控制信号进行了多次调试，培养了定位问题和优化设计的能力。4.最终通过仿真结果和上板测试，成功实现了单周期 CPU 对多条指令的正确处理，验证了设计的正确性和模块的协同工作能力。